



## Hareket Hissiyatı

| Parametreler |             |
|--------------|-------------|
| Sabit        | Tipik Değer |
| IVME         | 0.5-0.8     |
| MAX_HIZ      | 4-6         |
| SURTUNME     | 0.3-0.5     |
| YERCEKIMI    | 0.4-0.6     |
| ZIP_HIZI     | -10-(-12)   |
| COYOTE_SURE  | 5-8 kare    |
| BUFFER_SURE  | 5-8 kare    |

| İvme + Sürtünme                                                                                                                                                                                                                                                                                                                         |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>if tuslar[K_LEFT]:     self.vx -= IVME elif tuslar[K_RIGHT]:     self.vx += IVME else:     if self.vx &gt; 0:         self.vx = max(0,             self.vx - SURTUNME)     elif self.vx &lt; 0:         self.vx = min(0,             self.vx + SURTUNME) # Hız hız sınırı self.vx = max(-MAX_HIZ,     min(MAX_HIZ, self.vx))</pre> |  |

## Coyote Time

| Coyote Sayacı                                                                                                                                                                                                                                                                                                                |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre># Her kare: eski_yerde = self.yerde # ... carpsıma ... if eski_yerde and \     not self.yerde:     self.coyote_sayac = 6 elif self.coyote_sayac &gt; 0:     self.coyote_sayac -= 1  def zipla(self):     if self.yerde or \         self.coyote_sayac &gt; 0:         self.vy = -11         self.coyote_sayac = 0</pre> |  |

## Jump Buffer

| Buffer Sayacı                                                                                                                                                                                                                                                                                               |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre># KEYDOWN: def zipla_istegi(self):     self.buffer_sayac = 6  # Her kare: if self.buffer_sayac &gt; 0:     self.buffer_sayac -= 1 if self.buffer_sayac &gt; 0 and \     (self.yerde or     self.coyote_sayac &gt; 0):     self.vy = ZIP_HIZI     self.buffer_sayac = 0     self.coyote_sayac = 0</pre> |  |

## Variable + Double Jump

| Variable Jump                                                                                                                                                 |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre># KEYUP olayında: def tus_birakildi(self):     if self.zipliyor and \         self.vy &lt; 0:         self.vy *= 0.4         self.zipliyor = False</pre> |  |

| Double Jump                                                                                                                                                                                                   |  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre>MAX_ZIPLAMA = 2  # Yere deyince: if self.yerde:     self.kalan_ziplama = 2  # Ziplama: if self.kalan_ziplama &gt; 0:     self.vy = -11 if self.yerde \         else -9     self.kalan_ziplama -= 1</pre> |  |

## Çarpışma Yönleri

| 4 Yön + 4 Düzeltme |              |
|--------------------|--------------|
| Yön                | Düzeltilme   |
| Alt (vy>0)         | bottom = top |
| Üst (vy<0)         | top = bottom |
| Sağ (vx>0)         | right = left |
| Sol (vx<0)         | left = right |

## Tek Yönlü Platform

| One-Way + Drop                                                                                                                                                                                                                                                                                                                                                                |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre># Sadece üstten for p in tek_yonluler:     if self.rect.colliderect(p):         if self.vy &gt; 0 and \             eski_alt &lt;= p.top + 1:             self.rect.bottom = p.top             self.vy = 0             self.yerde = True  # Drop-through: DROP_SURE = 10 if asagi and space and self.yerde:     self.drop_sayac = DROP_SURE     self.yerde = False</pre> |  |

## Hareketli Platform

| Delta Transferi                                                                                                                                                                                                                                                                                                              |  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <pre># Platform: def guncelle(self):     eski = Vector2(         self.rect.topleft)     # ... hareket ...     self.delta = Vector2(         self.rect.topleft) - eski  # Oyuncu: if self.uzerindeki_platform:     p = self.uzerindeki_platform     self.rect.x += round(p.delta.x)     self.rect.y += round(p.delta.y)</pre> |  |

## Crumbling + Bouncy

| Crumbling FSM                                                                                                                                                                                             |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| sağlam (30 kare sarsılıyorsa) → yok (120 kare) → sağlam                                                                                                                                                   |  |
| Bouncy                                                                                                                                                                                                    |  |
| <pre>ZIPLAMA_GUCU = -18  # Üstten inis: if self.vy &gt; 0 and \     eski_alt &lt;= bp.top + 2:     self.rect.bottom = bp.top     self.vy = ZIPLAMA_GUCU     self.kalan_ziplama = 2     bp.tetikle()</pre> |  |

Dusman Temel (Sprite)

Dusman + Fizik

```
class Dusman(Sprite):
    def __init__(self, x, y, w, h):
        super().__init__()
        self.image = Surface((w, h))
        self.rect = self.image.get_rect(
            topleft=(x, y))
        self.vx = 0; self.vy = 0
        self.can = 1
        self.oldurulebilir = True

    def _fizik(self, katilar):
        # ... yatay + dikey ...
```

Patrol AI

Ucunum Tespiti

```
def _ucunum_var(self, katilar):
    if self.vx == 0:
        return False
    test_x = (self.rect.right+2
              if self.vx > 0
              else self.rect.left-6)
    test = Rect(test_x,
                self.rect.bottom+2, 4, 6)
    for k in katilar:
        if test.colliderect(k):
            return False
    return True # ucunum
```

Chase AI

Mesafe Kontrol

```
FARK_MESAFE = 200

a = Vector2(self.rect.center)
b = Vector2(oyuncu.rect.center)
yakın = a.distance_to(b) \
    <= FARK_MESAFE

# FSM gecis:
if yakın and self.durum == "patrol":
    self.durum = "chase"
```

Ranged AI + Mermi

Mermi Fırlat

```
a = Vector2(self.rect.center)
b = Vector2(oyuncu.rect.center)
yon = (b - a).normalize()
mermi = Mermi(a.x, a.y,
              yon.x * MERMI_HIZ,
              yon.y * MERMI_HIZ)
mermi_grubu.add(mermi)
```

Stomping

Dusman Ezme

```
for d in list(dusmanlar):
    if oyuncu.rect.colliderect(
        d.rect):
        if (oyuncu.vy > 0 and
            oyuncu.rect.bottom <
                d.rect.top + 16 and
            d.oldurulebilir):
            d.kill()
            oyuncu.vy = -9
        else:
            oyuncu.hasar_al(1)
```

PowerUp Eftkleri

Efekt Sözlüğü

```
self.efektler = {}
# Topla:
self.efektler["hiz"] = 600
# Her kare:
for t in list(
    self.efektler.keys()):
    self.efektler[t] -= 1
    if self.efektler[t] <= 0:
        del self.efektler[t]
# Kullan:
max_hiz = MAX_HIZ * (
    2 if "hiz" in self.efektler
    else 1)
```

I-Frames + Flicker

I-Frames

```
I_FRAME_SURE = 60

def hasar_al(self, m):
    if self.iframe_sayac > 0:
        return
    self.can -= m
    self.iframe_sayac = \
        I_FRAME_SURE

# Her kare:
if self.iframe_sayac > 0:
    self.iframe_sayac -= 1

On/Off Flicker

def ciz(self, ekran):
    if self.iframe_sayac > 0 \
        and (self.iframe_sayac
            // 4) % 2 == 0:
        return
    ekran.blit(self.image,
                self.rect)
```

Tile ID Şeması

| Bölüm 11 Şeması |              |
|-----------------|--------------|
| ID              | Tür          |
| 1               | boş          |
| 2               | solid taş    |
| 3               | oyuncu spawn |
| 4               | altın        |
| 5               | diken        |
| 6               | tek yönlü    |
| 7               | hareketli    |
| 8               | crumbling    |
| 9               | bouncy       |

Hata Önleme

| Sık Hatalar        |                 |
|--------------------|-----------------|
| Hata               | Çözüm           |
| Coyote sıfırlanmaz | = 0 zıpla sonu  |
| Tunnelling         | vy < TILE       |
| Platform kayma     | Delta transferi |
| Ucunumdan düşme    | Ayak altı testi |
| Chase'te duvar     | Zaman sayacı    |
| I-frames yok       | Hasar iptali    |